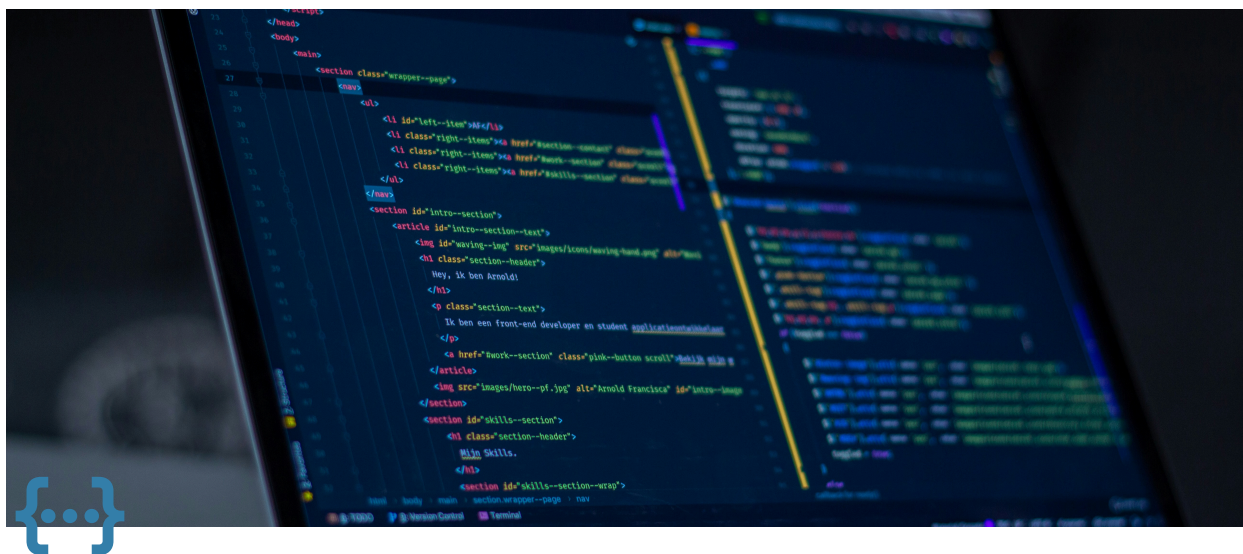




Javascript Data Type

◆ 1. What are Data Types?

Data types define **what kind of value a variable can store**.

In JavaScript, data types are divided into:

✓ Primitive Data Types

- String
- Number
- Boolean
- Undefined
- Null
- BigInt
- Symbol

✓ Non-Primitive (Reference Types)

- Object
- Array

- Function

◆ 2. Variables (How we store data)

```
let userName="Ashutosh";  
const age=22;  
var isLoggedIn=true;
```

👉 Best Practice:

- Use `let` → when value can change
- Use `const` → when value should NOT change (recommended most of the time)
- Avoid `var` (old, function-scoped, buggy)

◆ 3. typeof Operator

`typeof` is used to check the **data type of a value**

```
console.log(typeof"Hello");// string  
console.log(typeof10);// number  
console.log(typeoftrue);// boolean
```

◆ 4. Primitive Data Types (with examples)

1. String

```
const userName="Ashutosh";  
  
console.log(typeofuserName);// string
```

👉 Used for text data

2. Number

```
const price = 99.99;  
  
console.log(typeof price); // number
```

👉 Includes integers & decimals

3. Boolean

```
const isLoggedIn = true;  
  
console.log(typeof isLoggedIn); // boolean
```

👉 Used for conditions (true/false)

4. Undefined

```
let data;  
  
console.log(typeof data); // undefined
```

👉 Variable declared but not assigned

5. Null ⚠️ (Important Interview Question)

```
const user = null;  
  
console.log(typeof user); // object ❌ (this is a JS bug)
```

👉 Means "intentional empty value"

6. BigInt

```
const bigNumber=123456789012345678901234567890n;  
  
console.log(typeof bigNumber);// bigint
```

👉 Used for very large numbers

7. Symbol

```
const uniqueId=Symbol("id");  
  
console.log(typeof uniqueId);// symbol
```

👉 Used for unique values (rare but important)

◆ 5. Non-Primitive Data Types

1. Object

```
const user= {  
  name:"Ashutosh",  
  age:22  
};  
  
console.log(typeof user);// object
```

2. Array ⚠️

```
const fruits= ["apple", "banana"];  
  
console.log(typeof fruits);// object ❌
```

👉 To properly check array:


```
console.log(Array.isArray(fruits)); // true
```

3. Function

```
function greet() {  
  return "Hello";  
}
```

```
console.log(typeof greet); // function
```

◆ 6. Naming Conventions (VERY IMPORTANT 🚀)

✓ Use camelCase

```
let userName = "Ashu";  
let totalPrice = 1000;  
let isUserLoggedIn = true;
```

✗ Avoid bad naming

```
let x = "Ashu"; // ✗ unclear  
let data1 = 100; // ✗ not meaningful
```

✓ Constants in UPPER_CASE

```
const API_URL = "https://api.example.com";  
const MAX_USERS = 100;
```

✓ Boolean Naming (very important)

```
letisLoggedIn=true;  
lethasAccess=false;  
letcanEdit=true;
```

✓ Function Naming

```
functiongetUserData() {}  
functioncalculateTotal() {}  
functionhandleSubmit() {}
```

👉 Always use **verb + noun**

🔥 7. Real-World Example

```
constuserName="Ashutosh";// string  
constage=22;// number  
constisPremiumUser=true;// boolean  
consthobbies= ["coding","reading");// array  
constprofile= { city:"Patna" };// object  
  
console.log(typeofuserName);// string  
console.log(typeofage);// number  
console.log(typeofhobbies);// object  
console.log(Array.isArray(hobbies));// true
```

⚡ Key Takeaways (Interview Ready)

- JS has **7 primitive types**
- `typeof null === "object"` → known bug

- Arrays are **objects**, use `Array.isArray()`
- Prefer `const` > `let` > avoid `var`
- Naming = **clean code** = **senior dev mindset**